Penerapan Arsitektur Serverless dan Service Level Objective pada Sistem Pencatatan Praktikum Laboratorium di Google Cloud

Muhammad Abrar Rayhan
[Program studi, institusi]
[alamat surel penulis]
https://github.com/Muhammadabrarrayhan2/labtrack
https://labtrack-api-b26757zszq-et.a.run.app/app/

Abstrak

Laboratorium komputer di perguruan tinggi umumnya mencatat kehadiran praktikum dan laporan mahasiswa dengan lembar kerja yang tersebar di banyak asisten, sehingga rekap kehadiran sulit dibuat tepat waktu. Penelitian ini membangun LabTrack, layanan web pencatatan praktikum yang berjalan di Google Cloud dengan Cloud Run sebagai platform komputasi tanpa server, Cloud SQL PostgreSQL sebagai basis data, dan Cloud Storage untuk berkas laporan. Seluruh infrastruktur ditulis dalam Terraform dan dirilis melalui GitHub Actions dengan Workload Identity Federation. Keandalan diukur dengan dua *service level objective* (SLO): ketersediaan 99,5% dan latensi p95 di bawah 800 ms. Pengujian dilakukan pada 26 Oktober 2026 dan mencakup uji integrasi, uji beban, uji stress, simulasi kegagalan basis data, serta uji pemulihan cadangan. Hasilnya, 50 pengguna serentak selama 5 menit menghasilkan 7.049 permintaan tanpa kegagalan dengan p95 69 ms. Layanan masih memenuhi target latensi pada 150 pengguna (p95 466 ms) dan jenuh di antara 150 dan 200 pengguna karena batas tiga instans, bukan karena basis data (CPU Cloud SQL maksimal 37,5%). Simulasi henti basis data menghasilkan gangguan sekitar 20 menit dan mengungkap bahwa pemeriksaan kesehatan yang bergantung pada basis data memperlambat pemulihan. Ketersediaan berbasis permintaan pada hari pengujian mencapai 99,857%. Perkiraan biaya operasional sekitar Rp160.000 sampai Rp240.000 per bulan, lebih rendah dari estimasi server di lokasi sendiri.

Kata kunci: komputasi awan, serverless, Cloud Run, service level objective, infrastruktur sebagai kode, sistem informasi laboratorium

Abstract

University computer laboratories often track practicum attendance and student reports in spreadsheets kept by individual teaching assistants, which delays attendance summaries. This study builds LabTrack, a practicum record service on Google Cloud using Cloud Run, Cloud SQL for PostgreSQL and Cloud Storage. All infrastructure is defined in

Terraform and released through GitHub Actions with Workload Identity Federation. Reliability is measured with two service level objectives: 99.5% availability and p95 latency below 800 ms. Tests on 26 October 2026 covered integration, load, stress, a database outage drill and a backup restore. With 50 concurrent users for 5 minutes the service handled 7,049 requests with no failures and a p95 of 69 ms. It met the latency target up to 150 users and saturated between 150 and 200 users because of the three instance cap, while database CPU peaked at 37.5%. The outage drill caused about 20 minutes of disruption and showed that a database dependent health check slows recovery. Request based availability for the test day was 99.857%.

Keywords: cloud computing, serverless, Cloud Run, service level objective, infrastructure as code, laboratory information system

1. Pendahuluan

Kegiatan praktikum menghasilkan dua jenis data yang terus bertambah sepanjang semester: catatan kehadiran per sesi dan laporan praktikum per mahasiswa. Di banyak laboratorium, data ini dikelola oleh asisten yang berbeda untuk setiap kelas. Koordinator yang ingin mengetahui mahasiswa dengan kehadiran di bawah syarat ujian harus mengumpulkan berkas dari beberapa orang lalu menggabungkannya sendiri. Proses ini lambat dan rawan salah salin.

Sistem informasi untuk masalah semacam ini secara fungsional tidak rumit. Tantangannya ada di sisi operasional. Laboratorium kampus jarang punya staf infrastruktur khusus, anggarannya terbatas, dan bebannya tidak merata: ramai pada jam praktikum, hampir nol pada malam hari dan masa libur. Server fisik yang dibeli untuk beban puncak akan lebih sering menganggur. Karakteristik beban seperti ini cocok dengan model komputasi awan yang ditagih berdasarkan pemakaian [1], [2].

Memindahkan aplikasi ke awan tidak otomatis membuatnya andal. Pengelola tetap perlu tahu seberapa andal layanan yang mereka jalankan dan apa yang terjadi saat komponennya gagal. Praktik *site reliability engineering* menawarkan cara mengukurnya dengan *service level indicator* (SLI), *service level objective* (SLO), dan anggaran galat [3], [4]. Penelitian ini menerapkan praktik tersebut pada skala kecil yang realistis untuk laboratorium.

Kontribusi penelitian ini ada tiga. Pertama, rancangan arsitektur serverless untuk sistem pencatatan praktikum yang seluruh infrastrukturnya bisa dibangun ulang dari kode. Kedua, pengukuran kinerja dan kapasitas di lingkungan Google Cloud sungguhan, termasuk titik jenuhnya. Ketiga, temuan dari simulasi kegagalan basis data yang relevan bagi siapa pun yang memakai pemeriksaan kesehatan di Cloud Run.

2. Tinjauan pustaka

2.1 Komputasi awan dan model layanan

NIST mendefinisikan komputasi awan sebagai model yang memberi akses jaringan sesuai permintaan ke kumpulan sumber daya komputasi yang dapat dikonfigurasi, disediakan dan dilepas dengan cepat dengan upaya pengelolaan minimal [1]. Definisi ini membagi layanan menjadi tiga model: *Infrastructure as a Service* (IaaS), *Platform as a Service* (PaaS), dan *Software as a Service* (SaaS). Armbrust dkk. [2] menekankan bahwa keuntungan ekonomi awan berasal dari elastisitas dan penagihan per pemakaian, yang paling terasa pada beban yang berfluktuasi.

Komputasi tanpa server (*serverless*) memperluas model PaaS. Pengembang mengirim kode atau kontainer, sedangkan penyedia menangani penskalaan termasuk turun ke nol instans. Cloud Run adalah layanan serverless berbasis kontainer di Google Cloud yang menskalakan instans berdasarkan jumlah permintaan serentak dan menagih berdasarkan waktu pemrosesan [5].

2.2 Infrastruktur sebagai kode dan pengiriman berkelanjutan

Infrastruktur sebagai kode memperlakukan konfigurasi server, jaringan, dan layanan sebagai berkas teks yang disimpan di sistem kendali versi, ditinjau, dan dijalankan secara otomatis [6]. Dengan cara ini lingkungan bisa dibangun ulang secara konsisten dan setiap perubahan tercatat. Pengiriman berkelanjutan (*continuous delivery*) melengkapinya dengan pipeline yang menguji setiap perubahan sebelum dirilis [7].

2.3 Keandalan dan SLO

SLI adalah ukuran kuantitatif dari aspek layanan, misalnya proporsi permintaan yang berhasil. SLO adalah target untuk SLI dalam jendela waktu tertentu. Selisih antara 100% dan target SLO disebut anggaran galat, yaitu jumlah kegagalan yang masih bisa diterima sebelum tim harus mengutamakan keandalan di atas fitur baru [3]. Beyer dkk. [4] menyarankan alert berbasis laju pembakaran anggaran galat (*burn rate*) karena lebih sedikit alarm palsu dibanding alert berbasis ambang sederhana.

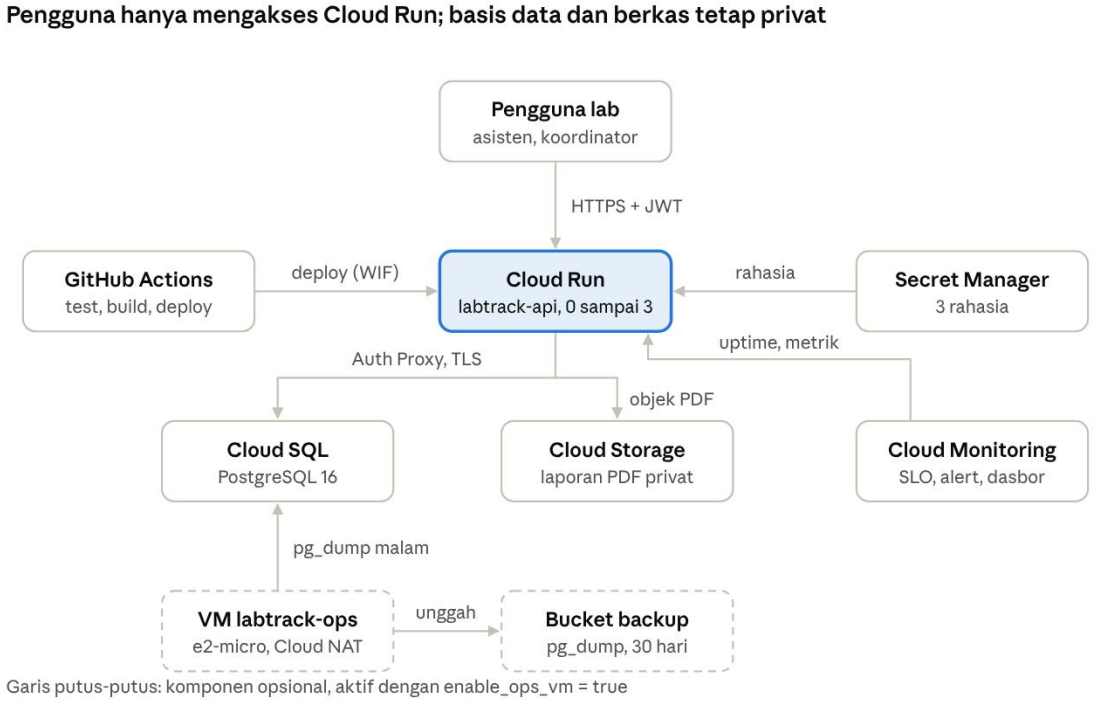
2.4 Standar kompetensi

Penelitian ini juga disusun sebagai bukti portofolio untuk skema Cloud Computing Engineer berdasarkan SKKNI Nomor 456 Tahun 2015 [8]. Unit yang relevan meliputi konsep dan teknologi komputasi awan, kebutuhan perangkat keras, aktivitas rutin pada IaaS, PaaS, dan SaaS, pemantauan sesuai SLA, serta dukungan teknis.

3. Metode penelitian

3.1 Tahapan

Penelitian mengikuti lima tahap: analisis kebutuhan bersama skenario penggunaan laboratorium, perancangan arsitektur dan model data, implementasi aplikasi dan infrastruktur, deployment ke Google Cloud, lalu evaluasi. Evaluasi terdiri atas uji fungsional otomatis, uji integrasi dari luar jaringan Google, uji beban, uji stress, simulasi insiden, dan uji pemulihan cadangan.



Gambar 1. Arsitektur LabTrack di Google Cloud

3.2 Arsitektur sistem

Aplikasi ditulis dengan FastAPI dan SQLAlchemy, dikemas sebagai kontainer, dan dijalankan di Cloud Run region `asia-southeast2` (Jakarta) dengan 0 sampai 3 instans dan konkurensi 40 permintaan per instans. Basis data memakai Cloud SQL PostgreSQL 16 kelas `db-f1-micro` yang diakses melalui konektor Auth Proxy. Laporan PDF disimpan di bucket Cloud Storage privat dan diunduh lewat URL bertanda tangan yang berlaku 15 menit. Rahasia (kata sandi basis data, kunci JWT, kata sandi admin awal) disimpan di Secret Manager. Antarmuka web berupa berkas statis yang dilayani kontainer yang sama di jalur `/app/`.

Autentikasi memakai JWT [9] dengan masa berlaku 120 menit. Kata sandi disimpan dengan bcrypt [10]. Untuk mencegah enumerasi akun, aplikasi tetap menjalankan perbandingan bcrypt terhadap hash tiruan ketika email tidak terdaftar.

Infrastruktur didefinisikan dengan Terraform 1.9.8. Komponen berbayar yang tidak wajib, yaitu kebijakan alert SLO dan mesin virtual operasi, berada di balik variabel boolean yang secara bawaan bernilai `false`. Mesin virtual operasi berjenis e2-micro di region `us-central1` agar masuk kuota gratis, tanpa IP publik, dan menjalankan `pg_dump` setiap malam ke bucket cadangan yang hanya mengizinkan penulisan objek baru.

3.3 Definisi SLO

Tabel 1. SLO yang diterapkan

SLO	SLI	Target	Jendela
Ketersediaan	Permintaan bukan 5xx dibagi seluruh permintaan	99,5%	30 hari bergulir
Latensi	Permintaan dengan latensi di bawah 800 ms dibagi seluruh permintaan	95%	30 hari bergulir

Target 99,5% memberi anggaran henti sekitar 216 menit per 30 hari. Target ini dipilih karena basis data berjalan sebagai satu instans tanpa replika. Target 99,9% hanya menyisakan sekitar 43 menit, kurang dari waktu satu kali pemeliharaan atau penyalaan ulang Cloud SQL yang teramati dalam penelitian ini.

Kedua SLO dibuat sebagai sumber daya `google_monitoring_slo` di Terraform. Enam kebijakan alert melengkapinya: kegagalan uptime check, laju pembakaran anggaran galat ketersediaan lebih dari 10 kali dalam 1 jam, rasio galat 5xx di atas 5%, serta CPU, disk, dan jumlah koneksi Cloud SQL yang mendekati batas.

3.4 Skenario pengujian

Tabel 2. Skenario pengujian

Pengujian	Alat	Parameter
Uji otomatis	pytest, SQLite dan PostgreSQL 16	72 kasus uji di setiap *push*
Uji integrasi	Skrip Python dari luar Google Cloud	9 skenario, data dibersihkan setelah selesai
Uji beban	Locust	50 pengguna, 5 menit, campuran baca dan tulis
Uji stress	Locust dengan profil bertahap	100, 150, 200, 250, 300 pengguna, masing-masing 2 menit
Simulasi insiden	`gcloud sql instances patch --activation-policy`	Cloud SQL dihentikan lalu dinyalakan kembali
Uji pemulihan	`sql-restore-test.sh`	Cadangan dipulihkan ke basis data sementara, jumlah baris dibandingkan

Data metrik diambil dari Cloud Monitoring API. Ketersediaan dihitung dengan dua cara: berbasis permintaan (proporsi permintaan non-5xx) dan berbasis waktu (proporsi jendela 5 menit uptime check yang sehat).

4. Hasil dan pembahasan

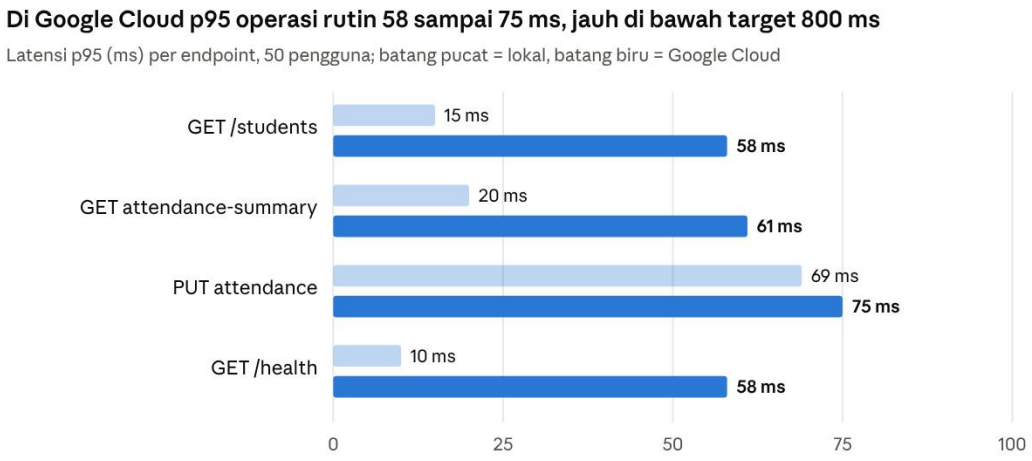
4.1 Deployment

Satu kali `terraform apply` pada proyek kosong membuat 62 sumber daya. Pipeline GitHub Actions menjalankan empat job, yaitu `test`, `test-postgres`, `terraform`, dan `deploy`, dan semuanya lulus. Durasi job `deploy` termasuk build citra dan pemeriksaan kesehatan adalah 1 menit 27 detik. Uji integrasi dari luar Google Cloud lulus pada sembilan dari sembilan skenario.

4.2 Kinerja pada beban normal

Tabel 3. Hasil uji beban 50 pengguna selama 5 menit

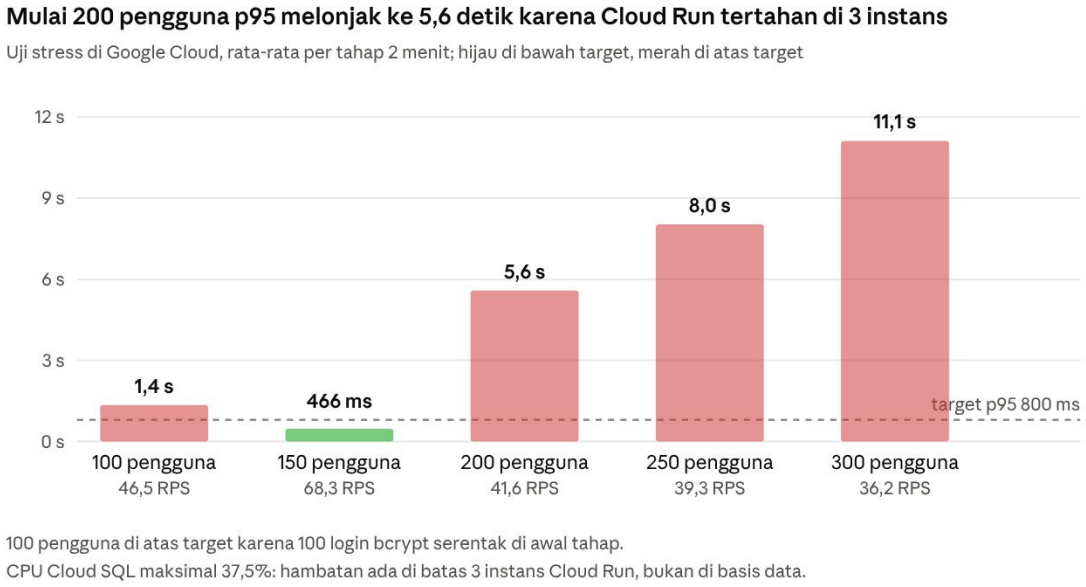
Besaran	Nilai
Total permintaan	7.049
Permintaan gagal	0
Throughput	23,58 permintaan per detik
Latensi p95 keseluruhan	69 ms



Gambar 2. Latensi p95 per endpoint: lingkungan lokal dibanding Google Cloud

Operasi rutin berada di 58 sampai 75 ms, kurang dari sepersepuluh target. Latensi di Google Cloud lebih tinggi daripada di lingkungan lokal, dengan selisih terbesar pada endpoint ringan seperti `/health` (10 ms lokal, 58 ms di awan). Selisih ini konsisten dengan tambahan waktu jaringan dari klien penguji dan lompatan melalui Auth Proxy, bukan karena pemrosesan aplikasi. Untuk endpoint yang lebih berat, seperti penulisan kehadiran, selisihnya hanya 6 ms.

4.3 Kapasitas dan titik jenuh

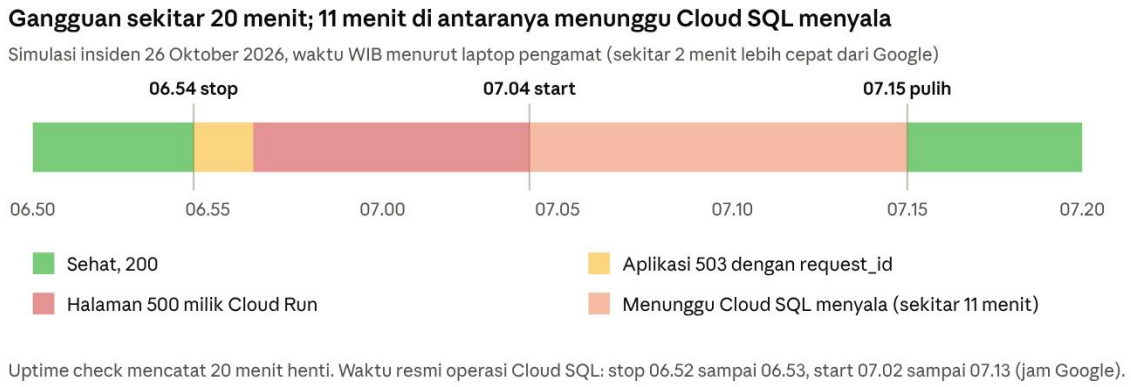


Gambar 3. Latensi p95 per tahap uji stress

Layanan memenuhi target latensi pada 150 pengguna dengan throughput tertinggi 68,3 permintaan per detik. Pada 200 pengguna p95 naik ke 5,6 detik dan throughput turun ke 41,6 permintaan per detik. Pola ini, yaitu latensi naik tajam sementara throughput justru turun, menandakan antrean di depan sumber daya yang sudah penuh. CPU Cloud SQL maksimal 37,5% sehingga basis data bukan penyebabnya. Penyebabnya adalah batas maksimum tiga instans Cloud Run, yang sengaja ditetapkan untuk menahan biaya dan menjaga jumlah koneksi basis data (3 instans dikali 7 koneksi, dari batas 25 pada `db-f1-micro`). Koneksi teramati mencapai 24 dari 25. Galat selama uji stress hanya 0,08%, seluruhnya HTTP 429 dari Cloud Run ketika antrean permintaan penuh.

Tahap 100 pengguna juga berada di atas target (p95 1,4 detik). Hal ini disebabkan 100 proses login serentak di awal tahap. bcrypt dirancang lambat secara sengaja [10], sehingga gelombang login yang bersamaan menahan CPU instans beberapa detik. Temuan ini praktis bagi laboratorium: lonjakan login di awal sesi praktikum adalah pola beban yang nyata, dan kapasitasnya perlu dihitung dari jumlah login serentak, bukan hanya jumlah pengguna aktif.

4.4 Simulasi kegagalan basis data



Gambar 4. Linimasa simulasi insiden 26 Oktober 2026

Cloud SQL dihentikan pukul 06.54 WIB. Pada fase pertama aplikasi menjawab 503 dengan `request_id`, sesuai rancangan. Pada fase berikutnya pengguna menerima halaman 500 bawaan Cloud Run tanpa `request_id`. Penelusuran log menunjukkan bahwa instans baru gagal melewati *startup probe*, karena probe memanggil `/health` yang ikut memeriksa koneksi basis data. Menyalakan kembali Cloud SQL membutuhkan sekitar 11 menit menurut log operasi Google. Total gangguan sekitar 20 menit, dan uptime check mencatat 20 menit henti.

Temuan ini menunjukkan risiko desain yang mudah terlewat. Pemeriksaan kesehatan yang terlalu "jujur" (ikut gagal ketika dependensi gagal) membuat platform menganggap kontainer rusak dan menolak menjalankannya, sehingga aplikasi kehilangan kemampuan memberi pesan galat yang informatif. Probe *startup* dan *liveness* sebaiknya hanya memeriksa proses aplikasi itu sendiri, sedangkan status dependensi dilaporkan lewat endpoint terpisah dan metrik.

4.5 Ketersediaan dan anggaran galat

Tabel 4. Realisasi SLO pada hari pengujian, 26 Oktober 2026

Indikator	Target	Realisasi
Ketersediaan berbasis permintaan	99,5%	99,857% (49 galat 5xx dari 34.342 permintaan)
Anggaran galat terpakai		28,5%
Proporsi permintaan di bawah 800 ms	95%	61,79%
Ketersediaan berbasis waktu	99,5%	94,521%

Dua cara menghitung ketersediaan memberi gambaran yang berbeda untuk hari yang sama. Berbasis permintaan, layanan memenuhi target karena selama henti basis data jumlah permintaan juga kecil. Berbasis waktu, 20 menit henti dalam jendela pengamatan yang pendek langsung menurunkan angka di bawah target. Untuk layanan internal dengan pengguna yang sedikit, metrik berbasis waktu lebih mencerminkan pengalaman

pengguna yang kebetulan mengakses saat gangguan. Sebaliknya, metrik berbasis permintaan lebih stabil untuk pelaporan bulanan. Laporan SLA yang dihasilkan otomatis oleh penelitian ini menampilkan keduanya.

Rendahnya proporsi permintaan di bawah 800 ms pada hari itu berasal dari uji stress yang memang dirancang untuk melewati kapasitas. Dalam operasional, kegiatan seperti ini dijadwalkan dan dikecualikan dari perhitungan SLA, tetapi data mentahnya tetap dilaporkan.

4.6 Biaya

Biaya terbesar adalah Cloud SQL, sekitar US$10 sampai 15 per bulan bila menyala terus. Cloud Run untuk beban laboratorium masih berada di kuota gratis karena turun ke nol instans di luar jam pakai. Total perkiraan Rp160.000 sampai Rp240.000 per bulan, dengan kurs contoh Rp16.000 per US$. Sebagai perbandingan, estimasi server di lokasi sendiri (harga pasar contoh) memerlukan biaya awal sekitar Rp30 juta dan biaya bulanan setara sekitar Rp1,29 juta setelah penyusutan, listrik, dan internet.

4.7 Keterbatasan

Pengujian dilakukan dalam satu hari dengan beban sintetis dari satu lokasi klien. Angka SLO bulanan yang sesungguhnya memerlukan pemakaian nyata selama minimal satu jendela 30 hari. Pengukuran pemulihan setelah uji stress dilakukan dalam sesi terpisah sekitar lima jam kemudian, sehingga waktu pemulihan langsung setelah beban puncak tidak terekam.

5. Kesimpulan

Arsitektur serverless dengan Cloud Run dan Cloud SQL dapat menjalankan sistem pencatatan praktikum dengan biaya di bawah Rp250.000 per bulan, latensi p95 69 ms pada 50 pengguna, dan kapasitas sekitar 150 pengguna serentak sebelum jenuh. Infrastruktur sebagai kode dan pipeline tanpa kunci rahasia membuat lingkungan bisa dibangun ulang dan diaudit. SLO dan anggaran galat memberi bahasa yang jelas untuk menilai keandalan. Simulasi insiden menunjukkan bahwa pemeriksaan kesehatan sebaiknya tidak bergantung pada basis data. Penelitian lanjutan dapat mengukur SLO selama satu semester penuh dengan pengguna nyata dan membandingkan konfigurasi Cloud SQL dengan IP privat dan replika baca.

Daftar pustaka

[1] P. Mell dan T. Grance, *The NIST Definition of Cloud Computing*, NIST Special Publication 800-145. Gaithersburg: National Institute of Standards and Technology, 2011.

[2] M. Armbrust dkk., "A view of cloud computing," *Communications of the ACM*, vol. 53, no. 4, hlm. 50-58, 2010.

[3] B. Beyer, C. Jones, J. Petoff, dan N. R. Murphy, *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol: O'Reilly Media, 2016.

[4] B. Beyer, N. R. Murphy, D. K. Rensin, K. Kawahara, dan S. Thorne, *The Site Reliability Workbook: Practical Ways to Implement SRE*. Sebastopol: O'Reilly Media, 2018.

[5] Google Cloud, "Cloud Run documentation," cloud.google.com/run/docs, diakses 26 Oktober 2026.

[6] K. Morris, *Infrastructure as Code: Dynamic Systems for the Cloud Age*, edisi ke-2. Sebastopol: O'Reilly Media, 2020.

[7] J. Humble dan D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston: Addison-Wesley, 2010.

[8] Kementerian Ketenagakerjaan Republik Indonesia, *Keputusan Menteri Ketenagakerjaan Nomor 456 Tahun 2015 tentang Penetapan SKKNI Kategori Informasi dan Komunikasi Golongan Pokok Aktivitas Pemrograman, Konsultasi Komputer dan Kegiatan YBDI Bidang Cloud Computing*. Jakarta, 2015.

[9] M. Jones, J. Bradley, dan N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, 2015.

[10] N. Provos dan D. Mazières, "A future-adaptable password scheme," dalam *Proceedings of the USENIX Annual Technical Conference, FREENIX Track*, 1999, hlm. 81-91.